

Control Flow in Angular

Control Flow in Angular (Angular 20+)

What Is Control Flow in Angular?

Control flow determines **which parts of a template are rendered and how many times**, based on conditions or collections.

In Angular 20+, control flow is implemented using **built-in control flow blocks**:

- @if
- @for
- @switch (with @case, @default)

These replace most use cases of traditional structural directives like:

- *ngIf
 - *ngFor
 - *ngSwitch
-

Why Angular Introduced New Control Flow

Older approach (*ngIf, *ngFor) had limitations:

- Heavy reliance on ng-template
- Indirect, desugared syntax
- Less readable for complex logic
- Runtime overhead

Angular 17+ (and fully recommended in Angular 20+) introduced **control flow blocks** that are:

- Compiler-native
- Block-based (like programming languages)
- Faster and more readable
- Signals-friendly

- Explicit (no hidden templates)

Control Flow Blocks in Angular 20+

Control Flow	Purpose	Replaces
@if	Conditional rendering	*ngIf
@for	Looping / iteration	*ngFor
@switch	Multi-branch logic	*ngSwitch

@if — Conditional Control Flow

Basic Syntax

```
@if (isLoggedIn) {  
  <p>Welcome back</p>  
}
```

With @else

```
@if (isLoggedIn) {  
  <p>Dashboard</p>  
} @else {  
  <p>Please login</p>  
}
```

With @else if

```
@if (role === 'admin') {  
  <p>Admin Panel</p>  
} @else if (role === 'user') {  
  <p>User Dashboard</p>  
} @else {
```

```
<p>Guest Access</p>
}
```

With Signals (Angular 20+ Best Practice)

```
isVisible = signal(true);
@if (isVisible()) {
  <p>Rendered using Signal</p>
}
```

@for — Iteration Control Flow

Basic Syntax

```
@for (product of products; track product.id) {
  <p>{{ product.name }}</p>
}
```

Why track Is Mandatory

- Prevents unnecessary DOM re-creation
 - Improves performance for large lists
 - Enforces best practices by design
-

Context Variables in @for

```
@for (user of users; track user.id; let i = $index) {
  <p>{{ i + 1 }} - {{ user.name }}</p>
}
```

Available variables:

Variable	Meaning
\$index	Current index

Variable	Meaning
\$first	First item
\$last	Last item
\$even	Even index
\$odd	Odd index
\$count	Total items

@empty Block (Very Important)

```
@for (item of items; track item.id) {
  <p>{{ item.name }}</p>
} @empty {
  <p>No records found</p>
}
```

This **eliminates extra @if checks** for empty lists.

@switch — Multi-Condition Control Flow

Syntax

```
@switch (status) {
  @case ('loading') {
    <p>Loading...</p>
  }
  @case ('success') {
    <p>Data loaded</p>
  }
  @case ('error') {
```

```
<p>Error occurred</p>
}
@default {
  <p>Unknown state</p>
}
}
```

When to Use

- Multiple exclusive conditions
 - Cleaner than chained @else if
 - Replaces *ngSwitch
-



Old vs New Control Flow Comparison

Aspect	Old (*ngIf, *ngFor)	New (@if, @for)
Syntax	Directive-based	Block-based
else if	Complex	Native
Empty state	Manual	@empty
Performance	Good	Better
Signals support	Indirect	Native
Recommendation	Maintenance	Preferred

Can We Still Use *ngIf and *ngFor?

Yes. Angular 20+ supports both.

But official guidance:

- **New development** → use control flow blocks
 - **Existing apps** → migrate gradually
-

Real-World Angular 20+ Example

```
@if (isLoading()) {  
  <app-spinner />  
} @else {  
  @for (product of products(); track product.id) {  
    <app-product-card [product]="product" />  
  } @empty {  
    <p>No products available</p>  
  }  
}
```

This pattern is now **idiomatic Angular**.

